TRANSFORMATION · DOMAIN 3 - DATA TRANSFORMATION, CLEANSING, QUALITY

Window functions, joins and aggregations

Section 3. The transformation surface, and the Databricks parts of it.

SECTION 3 of 10 WEIGHT 10% TOPIC Transformation

admin@sthithapragna.com

A group by **collapses** rows. A window function **keeps** them, and adds an answer computed across their neighbours.

A window specification has three parts

- **PARTITION BY** divides rows into independent groups. Calculations reset at each boundary.
- **ORDER BY** sets the order within a partition. Ranking and offsets are meaningless without it.
- **The frame clause** narrows to a sliding subset of the partition.
- Aliases exist: SORT BY for ORDER BY, DISTRIBUTE BY for PARTITION BY, and CLUSTER BY when there is no ORDER BY.

> ON THE EXAM
>
> Partition by does not collapse rows the way group by does. It only decides where the calculation restarts.

Three functions that look similar

FUNCTION	BEHAVIOUR ON TIES
`row_number`	Always distinct. Ties are broken arbitrarily. 1, 2, 3, 4.
`rank`	Ties share a rank, then a gap follows. 1, 2, 2, 4.
`dense_rank`	Ties share a rank, no gap follows. 1, 2, 2, 3.

ON THE EXAM

Row number is the one to reach for when deduplicating, precisely because it never ties. The other two are for genuine ranking.

Deduplicating with row number

- Partition by the **key that should be unique**, order by whatever decides which copy wins.
- Assign a row number, then keep only the rows numbered one.
- Ordering by a timestamp descending keeps the latest record per key.
- Ordering by nothing meaningful makes the choice arbitrary, and it will change between runs.

> ON THE EXAM
>
> This is deduplication that chooses which copy survives. Distinct cannot do that, because it has no way to prefer one row over another.

QUALIFY filters window results directly

- Window functions are evaluated **after** WHERE and HAVING, so you cannot filter on a rank there.
- **QUALIFY** filters window function results in the same query block. No subquery, no CTE.
- At least one window function must appear in the SELECT list or in the QUALIFY clause itself.
- It **cannot contain aggregate functions**, and it is not available in open-source Spark.

> ON THE EXAM
>
> Available from DBR 10.4 LTS. A genuinely Databricks-specific clause, which makes it a clean discriminator in a question.

ROWS against RANGE

	BEHAVIOUR
ROWS	Physical offsets. Counts a fixed number of rows before or after the current one.
RANGE	Logical offsets on the ORDER BY value. Rows sharing a value form one peer group.
Requirement	RANGE needs an ORDER BY with exactly one expression, or it errors.
The default	With ORDER BY and no explicit frame: RANGE, unbounded preceding to current row.

ON THE EXAM

The default being RANGE rather than ROWS is the detail that catches people. With duplicate ordering values, a running total jumps rather than climbing one row at a time.

Seven join types

TYPE	WHAT COMES BACK
INNER	Rows matching in both. This is the default when you write JOIN alone.
LEFT OUTER	All left rows, with nulls where the right has no match.
RIGHT OUTER	All right rows, with nulls where the left has no match.
FULL OUTER	All rows from both, nulls on whichever side is missing.
LEFT SEMI	Left rows that have a match. No columns from the right.
LEFT ANTI	Left rows that have no match.
CROSS	The Cartesian product. Every left row against every right row.

ON THE EXAM

Semi and anti are filters expressed as joins. Neither adds columns, and neither can multiply rows.

The filter that does not multiply rows

- An inner join used as a filter **duplicates left rows** when the right side has more than one match.
- A **semi join** returns each qualifying left row exactly once, whatever the right side contains.
- An **anti join** is the clean way to find what is missing, and it handles nulls better than NOT IN.
- Both express intent. The reader can see you wanted a filter and not an enrichment.

> ON THE EXAM
>
> A row count that grew after a join that was only supposed to filter is the signature of this mistake.

ON, USING, and the one to avoid

- **ON** takes any boolean expression, including inequalities and multiple conditions.
- **USING** names shared columns and returns them once rather than twice.
- **NATURAL** matches on **every** column with a shared name, implicitly.
- So adding a column upstream can silently change what a natural join matches on.

> ON THE EXAM
>
> Natural join is in the language and is a liability in a pipeline, because the join condition is decided by column names you do not control.

Beyond a plain group by

- **GROUPING SETS** computes several groupings in one pass, rather than unioning separate queries.
- **ROLLUP** adds subtotals up a hierarchy. **CUBE** adds every combination.
- A **FILTER** clause on an aggregate passes only matching rows to that function.
- That gives conditional aggregates without a case expression inside every function.

> ON THE EXAM
>
> One pass over the data instead of several. That is the efficiency the objective is asking about.

What changes when the input is unbounded

- Ranking across an entire partition needs to see every row, so it does not translate to an unbounded stream.
- Time-based windows plus a **watermark** are the streaming equivalent of a bounded aggregation.
- Chained stateful operations support **append output mode only**, from module 6.
- Deduplication has a streaming form, and it needs an event time column so state can be evicted.

> ON THE EXAM
>
> A pattern that works in batch is not automatically available in a stream. Anything that needs the whole partition is the first thing to check.

What to carry into the exam

- Group by collapses rows. A window function keeps them and adds context.
- **row_number** never ties, which is why it deduplicates. Rank gaps, dense rank does not.
- **QUALIFY** filters window results without a subquery. Databricks only, from DBR 10.4 LTS.
- The default frame is **RANGE**, not ROWS. Duplicate ordering values behave as one peer group.
- **Semi and anti joins** filter without adding columns or multiplying rows.

> ON THE EXAM
>
> When a question mentions efficiency, look for the option that reads the data once rather than the one that reads it three times.

> A bronze table receives the same order record multiple times when an upstream system retries. Each copy has an identical order id but a different ingestion timestamp, and later copies may correct earlier values.
>
> A silver job must keep exactly one row per order id, and it must be the most recent copy.

Which approach is correct?

A SELECT DISTINCT across all columns, which removes the duplicate order records.

B row_number over a partition by order id, ordered by ingestion timestamp descending, keeping row 1.

C GROUP BY order id with MAX on every other column.

D dense_rank over a partition by order id, keeping rank 1.

◆ PAUSE AND DECIDE

Which approach is correct?

A SELECT DISTINCT across all columns, which removes the duplicate order records.

The copies differ in timestamp and corrected values, so they are not identical rows. Distinct keeps them all.

B row_number over a partition by order id, ordered by ingestion timestamp descending, keeping row 1.

Partitions on the key that should be unique, orders so the latest wins, and row number guarantees exactly one.

C GROUP BY order id with MAX on every other column.

Takes the maximum of each column independently, producing a row that never existed.

D dense_rank over a partition by order id, keeping rank 1.

Dense rank ties. Two copies sharing a timestamp would both be kept.

ON THE EXAM

C is the dangerous one. It produces a plausible-looking row assembled from different source records, and nothing errors.

> A query must return, for each store, the three highest-value transactions. An engineer writes a ranking function in the SELECT list and then adds a WHERE clause filtering the rank to three or less.
>
> The query fails, saying the rank column cannot be resolved in the WHERE clause.

What is the cleanest fix on Databricks?

A Move the filter into a HAVING clause, which is evaluated later than WHERE.

B Use QUALIFY to filter the window function result in the same query block.

C Wrap the query in a CTE and filter in an outer SELECT.

D Add the rank expression to a GROUP BY so it becomes available to WHERE.

◆ PAUSE AND DECIDE

What is the cleanest fix on Databricks?

A Move the filter into a HAVING clause, which is evaluated later than WHERE.

HAVING filters groups after aggregation. Window functions are still evaluated after it.

B Use QUALIFY to filter the window function result in the same query block.

Exactly what QUALIFY exists for, and it avoids the extra query block entirely.

C Wrap the query in a CTE and filter in an outer SELECT.

Correct, and it is the workaround QUALIFY replaced. On Databricks it is no longer the cleanest option.

D Add the rank expression to a GROUP BY so it becomes available to WHERE.

Grouping by a rank collapses the rows you were trying to keep.

ON THE EXAM

C works and is what you would write in open-source Spark. The question asks for the cleanest fix on Databricks, and that word matters.

> A daily job filters a 200 million row transactions table down to those whose customer appears in a small compliance watchlist. The engineer uses an inner join on customer id.
>
> The output has more rows than the input, and finance reports inflated totals. The watchlist contains a few customers listed twice under different case references.

What is the correct fix?

A Deduplicate the watchlist, then keep the inner join.

B Replace the inner join with a LEFT SEMI join.

C Replace the inner join with a LEFT OUTER join and filter out nulls.

D Add SELECT DISTINCT to the output of the join.

◆ PAUSE AND DECIDE

What is the correct fix?

A Deduplicate the watchlist, then keep the inner join.

Fixes today's data and leaves the design fragile. The next duplicate reintroduces the bug.

B Replace the inner join with a LEFT SEMI join.

A semi join returns each qualifying left row once regardless of how many matches exist on the right.

C Replace the inner join with a LEFT OUTER join and filter out nulls.

Still multiplies rows on duplicate matches, and does more work first.

D Add SELECT DISTINCT to the output of the join.

Masks the duplication at the cost of a shuffle over 200 million rows, and would collapse legitimate identical transactions.

ON THE EXAM

The intent was to filter and the tool multiplied. Choosing a join that cannot multiply removes the whole class of bug.

A report computes a running total of daily revenue, ordered by date. Several dates share the same revenue value in the test data.

The engineer wrote an ORDER BY with no explicit frame clause, and finds that the running total jumps by several days' worth at once rather than increasing one day at a time.

What explains this?

A The default frame is RANGE, so rows sharing an ordering value are treated as one peer group.

B The ORDER BY is missing a PARTITION BY, so all rows are in one partition.

C Running totals require dense_rank rather than a sum over a window.

D The frame defaults to UNBOUNDED FOLLOWING, so future rows are included.

◆ PAUSE AND DECIDE

What explains this?

A The default frame is RANGE, so rows sharing an ordering value are treated as one peer group.

With ORDER BY and no frame, the default is RANGE unbounded preceding to current row. Peers are included together.

B The ORDER BY is missing a PARTITION BY, so all rows are in one partition.

A running total across all dates is what was wanted. That is not the symptom.

C Running totals require dense_rank rather than a sum over a window.

Ranking functions do not compute totals.

D The frame defaults to UNBOUNDED FOLLOWING, so future rows are included.

The default stops at the current row, not at the end of the partition.

ON THE EXAM

Switching the frame to ROWS gives the row-at-a-time behaviour that was expected. The default is the surprise, not the data.

> An analytics team needs revenue totals by region, by region and product, by product alone, and a grand total. They currently run four separate queries against the same 4 billion row table and union the results.
>
> The job takes hours and the cost has been questioned.

What is the most efficient rewrite?

A Use GROUPING SETS to compute all four groupings in a single pass.

B Cache the table, then run the four queries against the cached copy.

C Use four window functions with different PARTITION BY clauses.

D Create four materialized views, one per grouping.

◆ PAUSE AND DECIDE

What is the most efficient rewrite?

A Use GROUPING SETS to compute all four groupings in a single pass.

One read of the table instead of four, which is precisely what the extension exists for.

B Cache the table, then run the four queries against the cached copy.

Four billion rows will not sit comfortably in cache, and it still scans four times.

C Use four window functions with different PARTITION BY clauses.

Window functions keep every row. This needs collapsed aggregates.

D Create four materialized views, one per grouping.

Helps repeated reads and still computes four separate aggregations.

ON THE EXAM

ROLLUP would also work if the groupings formed a strict hierarchy. They do not here, because product alone is not under region.

> A batch job identifies each customer's single most recent order using a ranking function partitioned by customer. The team now wants the same logic in a Structured Streaming pipeline over an unbounded order feed.
>
> They report that the streaming version will not run.

What is the correct assessment?

A. Ranking across a whole partition needs every row, so it does not translate to an unbounded stream.

B. Streaming supports ranking only if the output mode is complete.

C. The ranking function must be replaced with QUALIFY, which is streaming-compatible.

D. Ranking works in streaming provided a checkpoint location is configured.

◆ PAUSE AND DECIDE

What is the correct assessment?

A Ranking across a whole partition needs every row, so it does not translate to an unbounded stream.

The stream never ends, so the engine can never know which order is last. AUTO CDC or a stateful operator is the route.

B Streaming supports ranking only if the output mode is complete.

Complete mode does not make an unbounded partition finite.

C The ranking function must be replaced with QUALIFY, which is streaming-compatible.

QUALIFY filters window results. It does not change what a window can compute.

D Ranking works in streaming provided a checkpoint location is configured.

A checkpoint records progress. It does not bound a partition.

ON THE EXAM

The real answer for 'latest record per key' on a stream is AUTO CDC from module 8, which is built for exactly this.